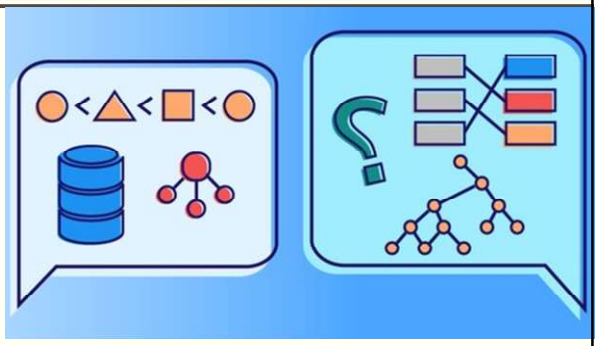
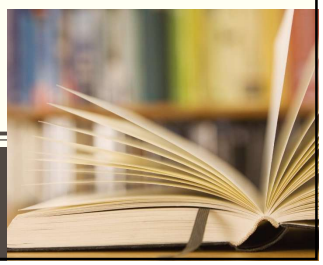
ALGORITMOS Y ESTRUCTURAS DE DATOS



Comisión F – 2026 – Cursado Anual



1

Algoritmos y Estructuras de Datos – AEDD – Agenda 19/10

- Datos en memoria Externa: Archivos.
- Archivos. Biblioteca fstream
- Tipos de Archivos.
- Archivos de texto.
- Archivos binarios.
- Clasificación según el tipo de acceso.
- Lectura y escritura de archivos secuenciales.
- Diferencias entre get y getline.
- Archivos binarios: función de lectura
- Archivos binarios: Función de escritura.

2

Algoritmos y Estructuras de Datos – AEDD – Asistencia 19/10

Contraseña del estudiante

i03f33

3

Datos en Memoria Externa: Archivos

Un archivo es una colección de datos almacenados juntos bajo un nombre común.

- Los programas en C++ son ejemplos de archivos.
- Los archivos almacenan información de manera permanente en dispositivos de almacenamiento de memoria secundaria, tales como dispositivos magnéticos (discos duros, cintas), discos ópticos (CDROM, DVD), memorias permanentes de estado sólido (memorias flash USB), etc.
- La información puede ser tanto programas (software) como datos que serán utilizados por los programas.
- Los archivos de datos pueden ser creados, leídos, actualizados y procesados por programas en C++.

4

Archivos

- **Según donde se almacenan**
  - **Internas** (en memoria principal)
  - **Externas** (en memoria auxiliar)
- **Según tipos de datos de sus componentes**
  - **Homogéneas** (todas del mismo tipo)
  - **No homogéneas** (pueden ser de distinto tipo)
- **Según la implementación**
  - **Provistas por los lenguajes** (básicas)
  - **Abstractas** (TDA - Tipo de dato abstracto que puede implementarse de diferentes formas)
- **Según la forma de almacenamiento**
  - **Estáticas** (ocupan posiciones fijas y su tamaño nunca varía durante todo el módulo)
  - **Dinámicas** (su tamaño varía durante el módulo y sus posiciones también)

5

Archivos

- Para tratar con un archivo se utilizan unas operaciones básicas que son: abrir, cerrar, escribir y leer.
- Los archivos pueden ser de lectura, en cuyo caso diremos que el tipo de acceso del archivo es de entrada, o bien de escritura, con lo cual tendremos un archivo de salida.
- Los archivos de salida pueden crearse (o sobreescribirse, si ya existen) o bien actualizarse.
- También se dispone de archivos de entrada / salida.
- El tipo de acceso determinará las operaciones disponibles para el archivo.

6

Biblioteca fstream

- La biblioteca que permite la manipulación de archivos recibe el nombre de **fstream**.
- Esta biblioteca nos provee de objetos que serán de **entrada (ifstream)** o de **salida (ofstream).**
- Todos los archivos disponen de operaciones para abrirlos, cerrarlos y preguntar si estamos al final del mismo.

Información relativa a un archivo

- Los objetos del tipo archivo son objetos lógicos (variables de nuestro programa) que representan un archivo físico.
- Para poder identificar de forma unívoca el archivo físico que manejan, se necesita un nombre de archivo.

7

Archivos y flujos



C++ considera a cada archivo como una secuencia de bytes o flujo. Un flujo transmite bytes en un solo sentido entre una fuente y un destino.
Cada archivo termina con un marcador de fin de archivo (EOF).

8

Tipos de Archivos

Existen dos tipos de archivos, que se distinguen de acuerdo a la codificación o formato en que almacenan la información:

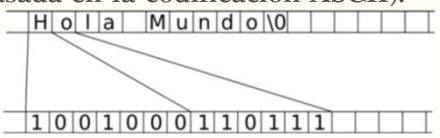
1. **Archivos de texto**
2. **Archivos binarios**

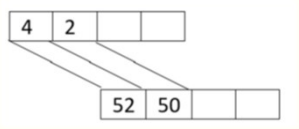
Ambos tipos de archivos almacenan datos usando un código binario; la diferencia está en lo que representan los códigos.

9

Archivos de texto

- La información se almacena como una secuencia de caracteres.
- Cada carácter individual (como una letra, dígito, signo de pesos, punto decimal, etc.) se almacena utilizando una codificación estándar (usualmente basada en la codificación ASCII).





- Al tratarse de un formato estandarizado, otros programas diferentes de aquel que creó el archivo (un procesador de palabras o un editor de texto) podrán entender y procesar su contenido.
- En el caso del software, los programas en código fuente codificados en un lenguaje de programación suelen ser almacenados como archivos de texto.

10

Archivo Binario

- La información se almacena con el mismo formato y codificación utilizada por el compilador C++ para almacenar sus datos primarios en memoria RAM:
  - Los números aparecen en su forma binaria verdadera
  - Las cadenas conservan su forma ASCII
- La ventaja de los archivos binarios es su compactibilidad: usan menos espacio para almacenar más números utilizando su código binario en lugar de valores de caracteres individuales.
- Están concebidos para ser procesados automáticamente por programas que conocen su formato interno.
- El procesamiento de archivos binarios es más eficiente que el de archivos de texto porque la información está representada directamente en código binario (exactamente tal y como se encuentra internamente en la memoria principal). De esta forma se evita la pérdida de tiempo que ocasionaría su conversión a un formato estándar (ASCII, UTF-8, etc), como ocurre con los archivos de texto.

Archivos Binarios

Si queremos guardar el número 123456 en un archivo de texto, cada dígito ocupará 8 bytes. Mientras que, usando un formato binario, el número 123456 se escribirá: 11110001001000000, lo que ocupará 4 bytes (es decir, lo que ocupa un int en C++).

11

Clasificación según el tipo de acceso

Por la manera de acceder a los archivos, los podemos clasificar en función de:

1. **La dirección del flujo de los datos:**
   - De entrada: Son aquellos cuyos datos se leen por parte del programa.
   - De salida: Aquellos archivos que el programa escribe.
   - De entrada/salida: Archivos que se pueden leer y escribir.
2. **Dependiendo de cómo se accede a los datos en sí:**
   - Secuencial: El orden de acceso a los datos está determinado, primero se accede al primer elemento y luego se puede ir accediendo a los siguientes, de uno en uno.
   - Directo / Aleatorio: Se puede acceder de forma directa a un elemento concreto del archivo. En este caso, el acceso es similar al de los arreglos.

12

Entrada y salida de información

- Un programa codificado en C++ realiza la entrada y salida de información a través de flujos de entrada y salida respectivamente.
- La entrada y salida de datos es a través de los flujos estándares de entrada y salida (cin y cout, respectivamente), usualmente conectados con el teclado y la pantalla de la consola.
- Todo lo visto respecto a la entrada y salida básica con los flujos estándares, o la entrada y salida de cadenas de caracteres, también es aplicable a los flujos de entrada y salida vinculados a archivos.



13

Archivo de Entrada/Archivo de Salida

Para determinar si un archivo es de entrada o salida se lo tiene que indicar a la hora de crearlo.

- Para crear un archivo de lectura (entrada) tenemos que declarar una variable del tipo ifstream. La operación que necesitaremos será la de leer elementos.

ifstream nombre-de-variable;

- Para crear un fichero de escritura (salida) declararemos una variable del tipo ofstream y precisaremos de una operación para escribir datos.

ofstream nombre-de-variable;

14

Ejemplo 1: Archivo de texto de entrada

```
#include <iostream>
#include <fstream>

using namespace std;

int main() {
    char z; int a; double c;

    ifstream f;
    f.open ("archi.txt");

    cout << "Lectura de un archivo" << endl;
    f >> z >> a >> c;

    cout << "Salida: " << endl;
    cout << z << "  " << a << "  " << c;

    f.close( );
    return 0;
}
```

archi: Bloc de notas

Archivo Edición Formato Ver Ayuda

h 23 45.7

Archivo de texto conteniendo un carácter, un valor entero y un valor double

Lectura desde el archivo

```
Lectura de un archivo
Salida:
h  23  45.7
```

15

Ejemplo 2: Archivo de texto de salida

```
#include <iostream>
#include <fstream>
using namespace std;

int main(void) {
    char z; int a; double c;
    z='h'; a=20; c=5.8;

    ofstream f;
    f.open ("sali.txt");

    cout << "Escritura en archivo" << endl;
    f << z << "  " << a << "  " << c;

    f.close();
    return 0;
}
```

```
Escritura en archivo
```

Escritura en el archivo

Archivo de texto generado

sali: Bloc de notas

Archivo Edición Formato Ver Ayuda

h 20 5.8

open: Cuando es para salida, si el archivo no existe lo crea; si ya existe lo sobreescribe vacío.

16

Ejemplo 3: Archivo con varios renglones

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    char z; int a; double c;

    ifstream f;
    f.open("archivo.txt");

    cout << "Lectura de archivo" << endl;
    f >> z >> a >> c;
    while (!f.eof()){
        cout << z << " " << a << " " << c << endl;
        f >> z >> a >> c;
    }

    f.close();
    return 0;
}
```

archivo.txt: Bloc de notas

Archivo Edición Formato Ver Ayuda

```
h   23   45.7
a   3    23.00
z   4    11.5
```

```
Lectura de archivo
h 23 45.7
a 3 23
z 4 11.5
```

17

Comprobar apertura de archivo

Un archivo físico que se haya intentado abrir y que no exista no tendrá su correspondiente archivo lógico asociado.

- Para comprobarlo se usa la instrucción fail que consulta si el archivo falló:

if (archivoTexto.fail())
// el archivo no se ha abierto

fail(): Devuelve true si el archivo no se ha abierto con éxito;
de lo contrario, devuelve false.

- O bien se puede hacer un if sobre la variable del tipo archivo:

if (!archivoTexto)
// el archivo no se ha abierto
Ambas posibilidades son equivalentes.

18

Lectura

- Una vez que el archivo está abierto, podemos leer elementos con el operador de entrada: “>>” .
- Se usa igual que en las instrucciones de entrada estándar (cin » variable) pero substituyendo el flujo cin por el nombre de la variable del tipo ifstream.
- Para leer un carácter: archivoTexto » caract;
- Dicha operación pone en la variable caract el siguiente elemento leído del archivo.
- Por defecto, la lectura con el operador “>>” ignora los espacios en blanco antes de un número o una cadena, las tabulaciones y los saltos de línea.
  - Para que no los ignore: archivoTexto.unsetf(ios::skipws);
  - Para que vuelva a ignorar los espacios en blanco: archivoTexto.setf(ios::skipws);

19

Ejemplo 4: Lectura de un archivo de texto

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    char caract;
    ifstream archivo_entr;
    archivo_entr.open("datos.txt");
    if (!archivo_entr)
        cout << "Error al abrir el fichero" << endl;
    else {
        archivo_entr.unsetf(ios::skipws);
        archivo_entr >> caract;
        while (!archivo_entr.eof()){
            cout << caract;
            archivo_entr >> caract;
        }
    }
    archivo_entr.close();
    return 0;
}
```

datos.txt: Bloc de notas

Archivo Edición Formato Ver

```
12 bc def ghi
ff
hh
b
```

```
12 bc def ghi
ff
hh
b
```

20

Lectura y escritura de archivos secuenciales

Cuando un programa quiere trabajar con un determinado archivo, debe realizar las siguientes acciones:

1. Incluir la biblioteca <fstream>, que contiene los elementos necesarios para procesar el archivo.
2. Usar el espacio de nombres std.
3. Declarar las variables que actuarán como manejadores de archivos.
4. Abrir el flujo de datos, vinculando la variable correspondiente con el archivo especificado. Esta operación establece un vínculo entre la variable (manejador de archivo) definida en nuestro programa y el archivo físico. Toda transferencia de información entre el programa y un archivo se realizará a través de la variable manejador que ha sido vinculada con dicho archivo.
5. Comprobar que la apertura del archivo se realizó correctamente. Si la vinculación con el archivo especificado no pudo realizarse por algún motivo (por ejemplo, si queremos hacer entrada de datos desde un archivo que no existe, o si es imposible crear un archivo en el que escribir datos), entonces la operación de apertura fallará.

21

Lectura y escritura de archivos secuenciales

6. Realizar la transferencia de información (de entrada o de salida) con el archivo a través de la variable de flujo vinculada al mismo.

   Normalmente, tanto la entrada como la salida de datos se realizan mediante un proceso iterativo. En el caso de entrada dicho proceso suele requerir la lectura de todo el contenido del archivo.
7. Comprobar que el procesamiento del archivo del paso previo se realizó correctamente. En el caso de procesamiento para entrada ello suele consistir en comprobar si el estado de la variable manejador indica que se ha alcanzado el final del archivo. El procesamiento para salida suele consistir en comprobar si el estado de la variable manejador indica que se ha producido un error de escritura.
8. Finalmente, cerrar el flujo para liberar la variable manejador de su vinculación con el archivo. En caso de no cerrar el flujo, éste será cerrado automáticamente cuando termine el ámbito de vida de la variable manejador del archivo.

22

Tratamiento de archivos

Todo tratamiento de archivos consta de tres pasos:

1. Apertura del archivo: Momento en el cual se mapea el objeto lógico (la variable que representa el archivo en nuestro programa) con el objeto físico (el archivo en disco). Del modo de apertura dependerá si un archivo es de lectura o escritura.

2. Acceso al archivo: Etapa en la cual se van leyendo o escribiendo los elementos que interesen.

3. Cierre del archivo: Actualiza el archivo y elimina la información relativa a él que se haya creado en el momento de abrirlo, tanto por parte del compilador como del sistema operativo.

Para abrir un archivo se usa la instrucción **open** y como parámetro se pasa el nombre del archivo y el modo de apertura.

Para cerrar un archivo se usa la instrucción **close** sin ningún argumento.

23

Opciones para abrir archivos

Para abrir un archivo usamos el método:

void open (const char nombre, int nModo = ios: :in)*

- nombre es el nombre del archivo
- nModo es el modo de apertura.

Modo	Significado
`ios::in`	Abre para lectura (valor por defecto para `ifstream`). Se sitúa al principio.
`ios::out`	Abre para crear nuevo archivo (valor por defecto para `ofstream`). Se sitúa al principio y pierde los datos.
`ios::app`	Abre para añadir al final.
`ios::trunc`	Crea un archivo para escribir/leer (si ya existe se pierden los datos).
`ios::binary`	Archivo binario.

24

GET

La instrucción ">>" se saltea (ignora) los espacios en blanco.

Si se requiere **leer** todo el archivo tal y como está **sin saltearse espacios, tabulaciones, etc.**, se puede usar la función **get**.

La operación get se puede llamar de 3 formas:

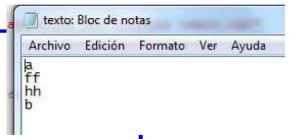
- **int get ( ):** Lee un sólo carácter y lo retorna. No se saltea espacios en blanco.
- **get(char& car)**: Lee un sólo carácter de un flujo de entrada y lo pone en la variable *car*.
- **get (char* cad, int numCar, char delimitador='\n'):** Se usa para leer un número determinado de caracteres indicado en *numCad* y termina si encuentra un salto de línea.
  - ✓ En este caso lo que se leerá es como máximo el número de caracteres indicado en *numCar-1* o el carácter delimitador, y los almacenará en la cadena de caracteres *cad*.
  - ✓ El carácter delimitador es opcional y sirve para definir un delimitador, que por defecto es el salto de línea. Si se encuentra el delimitador, **no es extraído** de la secuencia de entrada, y permanece ahí como el siguiente carácter a ser extraído.

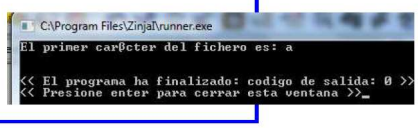
25

Ejemplo 5. Uso de get(char&)

```
#include <iostream>
#include <fstream>
using namespace std;

int main(){
    ifstream ficheroTexto;
    char c;
    ficheroTexto.open ("texto.txt");
    if (!ficheroTexto)
        cout << "Error al abrir el archivo" << endl;
    else {
        //Comprobamos que el fihero no esté vacío
        if (!ficheroTexto.eof()){
            ficheroTexto.get(c); //Leemos el 1er caracter
            cout << "El primer caracter del fichero es: "
                 << c << endl;
        }
    }
    ficheroTexto.close();
    return 0;
}
```





26

Ejemplo 6. Uso de get con parámetros

```
#include <iostream>
#include <fstream>
using namespace std;

int main(){
    ifstream ficheroTexto;
    char car[30];

    ficheroTexto.open ("texto.txt");
    if (!ficheroTexto)
        cout << "Error al abrir el archivo" << endl;
    else {
        //Comprobamos que el fihero no esté vacío
        if (!ficheroTexto.eof()){
            ficheroTexto.get(car,3,'\n');
            cout << "Los caracteres leidos del fichero son: "
            << car << endl;
        }
    }
    ficheroTexto.close();
    return 0;
}
```

texto.txt: Bloc de notas

Archivo Edición Formato Ver

abcd
ff
hh
b

C:\Program Files (x86)\Zinjal\bin\run

Los caracteres leidos del fichero son: ab

27

Lectura: getline

Permite la lectura de toda una línea.

Tiene la siguiente sintaxis:

getline(char* cad, int numCar, char delim = '\n');

Cuando se ejecuta almacena en la cadena cad los caracteres del archivo hasta que:

- encuentre el delimitador delim (indicado como 3er parámetro), o
- como máximo haya leído numCar - 1 caracteres, o
- encuentre el final del archivo

En el último elemento de la cadena cad se guarda el carácter final de cadena: '\0'.

El 3er parámetro sirve para definir un delimitador, que por defecto es el salto de línea. Si se encuentra el delimitador, se extrae de la secuencia de entrada pero es descartado, no se copia a la cadena cad.

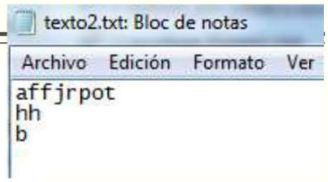
28

Ejemplo 7. Uso de getline

```
#include <iostream>
#include <fstream>
using namespace std;

int main(){
    ifstream ficheroTexto;
    char car[30];

    ficheroTexto.open ("texto.txt");
    if (!ficheroTexto)
        cout << "Error al abrir el archivo" << endl;
    else {
        if (!ficheroTexto.eof()){
            ficheroTexto.getline(car,30,'\n');
            cout << car << endl;
        }
    }
    ficheroTexto.close();
    return 0;
}
```




```
Los caracteres leidos del fichero son: affjrpot

<< El programa ha finalizado: codigo de salida: 0 >>
<< Presione enter para cerrar esta ventana >>
```

29

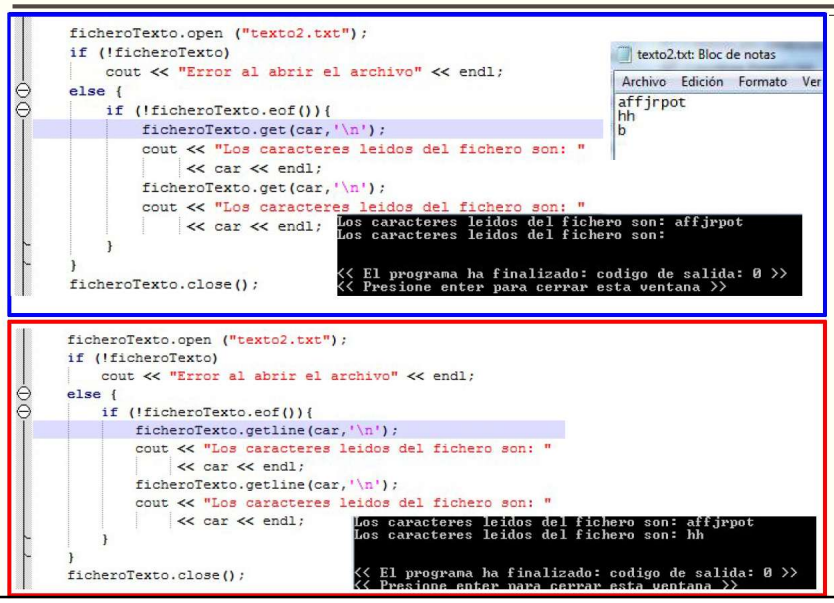
Diferencias entre get y getline

Hay una diferencia sutil, pero importante:

- La función get() se detiene cuando ve el delimitador en el flujo de entrada, pero no lo extrae del flujo de entrada. Entonces, si se hace otro get() usando el mismo delimitador, retornará inmediatamente sin ninguna entrada contenida.
- La función getline(), por el contrario, sí extrae el delimitador del flujo de entrada, pero no lo almacena en la cadena resultante.

30

Diferencias entre get y getline



31

Archivos de acceso directo o aleatorio

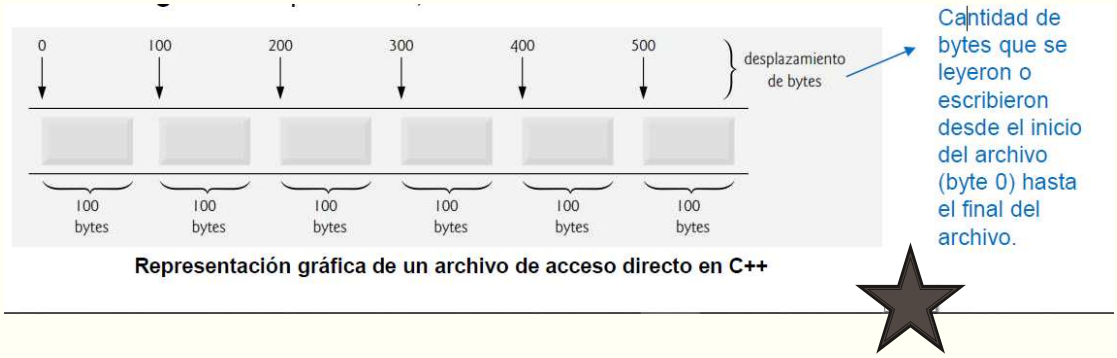
Los archivos secuenciales no son apropiados para las aplicaciones de acceso directo, en las que un registro específico se debe localizar inmediatamente: sistemas de reservación de aerolíneas, sistemas bancarios, sistemas de punto de venta, cajeros automáticos, etc.

Para estos casos se requiere de archivos de acceso aleatorio, cuyos registros individuales se pueden acceder de manera directa y rápida, sin tener que buscar en otros registros.

C++ no impone una estructura sobre un archivo de acceso aleatorio. La aplicación que desee utilizar este tipo de archivos debe crearlos. El método más sencillo es requerir que todos los registros en el archivo sean de la misma longitud fija. De este modo, es más fácil para el programa calcular la ubicación exacta de cualquier registro relativo al inicio del archivo facilitando el acceso inmediato a registros específicos, incluso en archivos extensos.

32

Archivos de acceso directo o aleatorio



Representación gráfica de un archivo de acceso directo en C++

33

Archivos binarios: la función de lectura

Prototipo: istream& read (char* s, streamsize n);

Lee un bloque de datos desde un archivo binario.

Extrae n caracteres (indicados en el 2do parámetro) desde un archivo de entrada y los almacena en la cadena de caracteres s (indicada en el 1er parámetro).

- ✓ Para leer un tipo de dato que no sea una cadena de caracteres se deberá hacer un casting.
- ✓ Esta función copia un bloque de datos sin chequear su contenido (si tiene caracters nulos o de final).

Cuando no sabemos la medida en bytes de la información que queremos leer, deberemos usar la función sizeof que dado un tipo nos devolverá su tamaño en bytes.

Ejemplo: Queremos leer de un archivo un valor de tipo double y guardarlo en una variable var de tipo double. Haremos:

```
archivo.read ( (char*)(& var), sizeof(double) );
```

34

Archivos binarios: la función de escritura

Prototipo: ostream& write (const char* s, streamsize n);

Escribe un bloque de datos en un archivo binario.

Inserta n caracteres (indicados en el 2do parámetro) en el archivo de salida, desde la cadena de caracteres s (indicada en el 1er parámetro).

✓Esta función tampoco valida contenido.

Ejemplo: Queremos escribir en un archivo un valor de tipo double que tenemos almacenado en una variable var de tipo double. Haremos:

archivo.write ((char)(& var), sizeof(double));*

35

Ejercicio 1

Mediante un programa C++, escribir un archivo binario llamado grupo.dat, formado por un conjunto de 200 pares de números generados aleatoriamente. Cada par de datos se conforma por un flotante y un entero.

36

Ejercicio 2

Escribir un programa que abra el archivo generado en el ejercicio anterior, y solicite al usuario que ingrese un flotante, un entero y una posición.

El programa debe sobreescribir el par en la posición ingresada por el usuario, por el nuevo par.

Luego, debe mostrar la lista de datos por pantalla mostrando un par por línea.

37

Algoritmos y Estructuras de Datos – AEDD – Asistencia 19/10

Contraseña del estudiante

i03f33

38

Para profundizar

Capítulo 17

"Procesamiento de archivos"

Libro: "Cómo programar en C++" – Deitel & Deitel,

pág. 708

Capítulo 8

"Flujos de archivos de E/S y archivos de datos"

Libro: "C++ para ingeniería y ciencias" 2da edición -

Gary J. Bronson, pág. 443

39

Algoritmos y Estructuras de Datos – AEDD – Agenda 19/10

- ✓ Datos en memoria Externa: Archivos.
- ✓ Archivos. Biblioteca fstream
- ✓ Tipos de Archivos.
- ✓ Archivos de texto.
- ✓ Archivos binarios.
- ✓ Clasificación según el tipo de acceso.
- ✓ Lectura y escritura de archivos secuenciales.
- ✓ Diferencias entre get y getline.
- ✓ Archivos binarios: función de lectura
- ✓ Archivos binarios: Función de escritura.

¡OBJETIVO CUMPLIDO!

40

Para profundizar

Capítulo 17

"Procesamiento de archivos"

Libro: "Cómo programar en C++" – Deitel & Deitel,

pág. 708

Capítulo 8

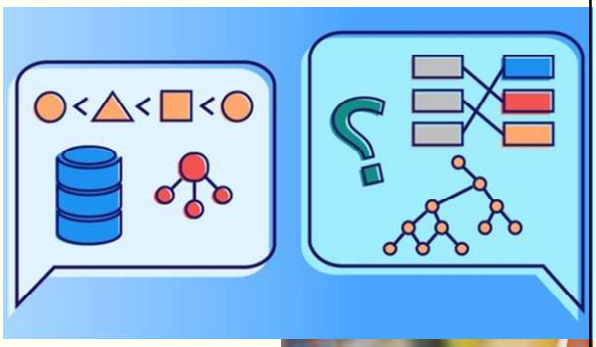
"Flujos de archivos de E/S y archivos de datos"

Libro: "C++ para ingeniería y ciencias" 2da edición -

Gary J. Bronson, pág. 443

41

ALGORITMOS Y ESTRUCTURA DE DATOS



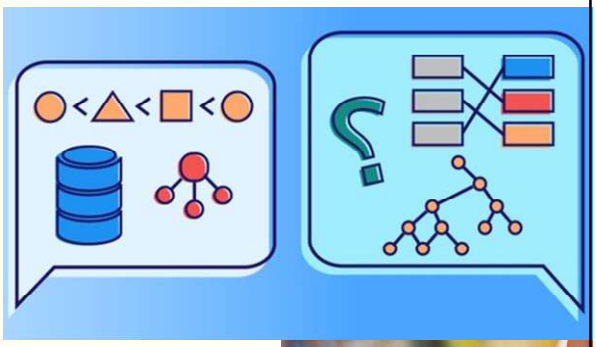
Comisión F – 2026 – Cursado Anual

¿DUDAS?



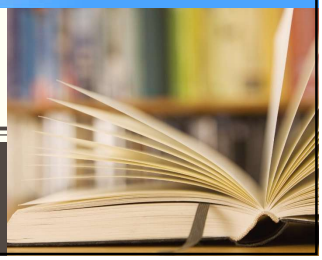
42

ALGORITMOS Y ESTRUCTURA DE DATOS



Comisión F – 2026 – Cursado Anual

¡Muy buena semana!



43